

Parcial - TAD Ronda (Lista Circular Doblemente Enlazada)

Un **TAD Ronda** es una estructura de datos que modela una **lista circular doblemente enlazada** de números enteros, diseñada para simular un proceso iterativo donde la posición de interés avanza cíclicamente. En todo momento, la estructura mantiene una referencia a un elemento llamado **"actual" (current)**. La estructura de nodos es **completamente cíclica** (el último nodo apunta al primero y viceversa).

Tipo Abstracto de Datos (TAD)

El TAD Ronda se define sobre la base de los siguientes tipos y primitivas:

<code>typedef struct RondaStr* Ronda;</code>	Representa la estructura de datos Ronda.
<code>typedef struct RondaNode* RondaNode;</code>	Representa un nodo individual de la lista.

Invariantes de Representación

Se debe garantizar que se cumplen los siguientes invariantes:

- **Para RondaNode:** Los punteros `next` (siguiente) y `prev` (previo) de un nodo **nunca** son `NULL` (en una lista no vacía) y siempre apuntan a otro nodo dentro de la estructura circular.
- **Para RondaStr (Ronda):**
 - Si la Ronda **no está vacía** (`current` no es `NULL`), la estructura de nodos debe formar un **ciclo cerrado y doblemente enlazado**.
 - Para cualquier nodo `N`, se cumple: `N -> next -> prev = N` y `N -> prev -> next = N`.

Se pide implementar las siguientes funciones, utilizando las estructuras `struct RondaNode` y `struct RondaStr`, garantizando el cumplimiento de los Invariantes de Representación.

<code>Ronda mkRonda();</code>	Crea una nueva Ronda vacía. $O(1)$.
<code>int current(Ronda ronda);</code>	Devuelve el valor del elemento actual de la Ronda. Precondición: La Ronda no está vacía. $O(1)$.

<code>void move(int pos, Ronda ronda);</code>	Mueve la posición actual <code>pos</code> veces en sentido horario (hacia <code>next</code>). La Ronda debe tener al menos un elemento. $O(pos)$.
<code>void insert(int value, Ronda ronda);</code>	Inserta un nuevo nodo con <code>value</code> después de la posición actual. El nuevo nodo insertado pasa a ser la nueva posición actual . $O(1)$.
<code>void remove(Ronda ronda);</code>	Elimina el nodo actual . El nuevo nodo actual pasa a ser el nodo previo al nodo eliminado. Si la Ronda queda vacía, <code>current</code> es <code>NULL</code> . Precondición: La Ronda no está vacía. $O(1)$.
<code>int length(Ronda ronda);</code>	Devuelve la cantidad de elementos en la Ronda. $O(n)$.

Ejercicios

1. Escribir los **Invariantes de Representación** para garantizar elementos válidos del TAD Ronda.
2. Implementar el TAD Ronda con sus primitivas, **tal como se describe arriba**.
3. Implementar las siguientes funciones de usuario que interactúan con el TAD Ronda. **DEBE** usar exclusivamente las primitivas del TAD definidas en la sección anterior (y `mkRonda`, si fuera necesario).

<code>int* toArray(Ronda ronda);</code>	Consume la Ronda, extrayendo todos sus elementos desde el actual hasta el final, y los devuelve en un arreglo de enteros. La Ronda queda vacía.
<code>Ronda fromArray(int len, int* arr);</code>	Construye una nueva Ronda a partir de los elementos de un arreglo <code>arr</code> de longitud <code>len</code> . El primer elemento del arreglo (<code>arr[0]</code>) debe quedar como el último nodo insertado (y por lo tanto, el nodo actual de la Ronda resultante).